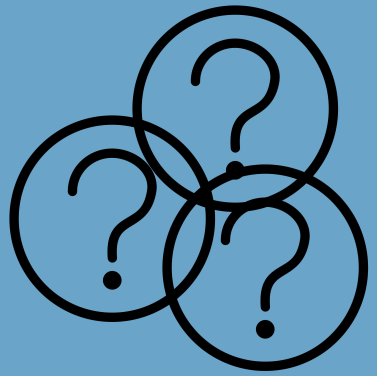
Text Processing

Attapol Thamrongrattanarit
NLP II 2026

What is a text? Why are we interested in texts?



Texts and corpora

- We want to study certain linguistic phenomena or create an NLP application. An important component is a corpus (plural = corpora).
- A corpus is a collection of texts produced specific group of authors of a specific language/dialect in a specific timeframe, in a specific place for a specific function.
- We sometimes call a corpus a dataset, which is broader.

What does text data look like?

Ah ! Petit prince, j'ai compris, peu à peu, ainsi ta petite vie mélancolique.
อา… เจ้าชายน้อย ผมค่อยเข้าใจถึงชีวิตอันหงอยเหงาเศร้าซึมของเธอแล้วละ
啊！小王子，我終於漸漸瞭解你那小小的身軀裏
Oh, little prince! Bit by bit I came to understand the secrets of your sad little life.
王子さま、あなたは、はればれしない日々を送ってこられたようだが

Issues with text data

- Noise from the data collection channel
- Variation of languages (sociolinguistic factors)
- Complexity of language

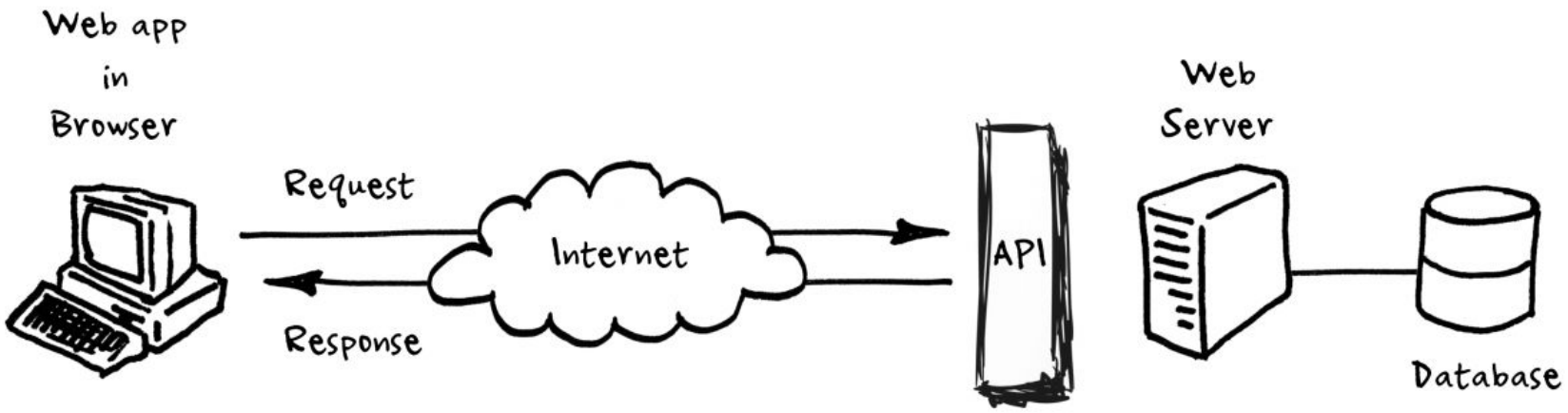
Issues with text data

- **Noise from the data collection channel**
- Variation of languages (sociolinguistic factors)
- Complexity of language

Where do texts/corpora come from?

- Text files
- PDF files
- The internet

API Call



https://www.boostit.net/blog/api-functionality-internet-services/sparkholder-api/

Activity: Inspect the dataset

- Explore (or 'eyeball') the dataset and see if there's something you need to do before analyzing the data
- https://github.com/luminati-io/Twitter-X-dataset-samples/blob/main/twitter-posts.csv

Data is dirty. Language data is even dirtier.

RT @MatichonOnline: “บิ๊กตู่”ลั่นรบ.ทำอะไรยึดกม.ไม่ใช่ติดคุกแล้วหนี เล่นมุกพรรคร่วม “พลังปชป.ภูมิใจไทย” https://t.co/9nmOBJnhrq via @มติชนอ...

บกพร่องโดยสุจริต VS อยู่-ไม่-เป็น | ขยี้คดีโกง | 10 พ.ย. 62 | (3/3) https://t.co/atUF6PrXdx via @YouTube

ขอให้นึกถึงการท่องเที่ยวภายภาคหน้าด้วยคะ

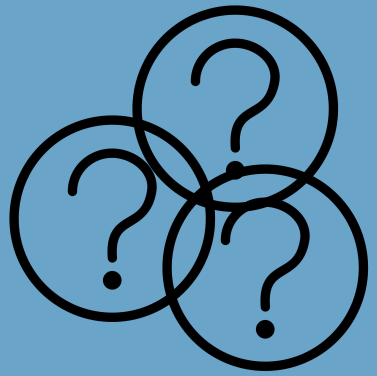
==========

เร่งสอบกลุ่ม<emph>คณะก้าวหน้า</emph> ภายในศุกร์นี้

What we do

- Clean out the non-text portion such as URL, #, numbers, HTML tags
- Decide whether you want other languages too
- Check the vocabulary to see if there is any remnants

What tool should we use to clean out the data?



Review: Regular Expression

Regular Expression

Regular expressions (often referred to as 'regex') are a language used to **specify and match patterns** of characters

Examples

- Searching for phone numbers that follow the pattern of having 10 digits, where the first digit must be 0 and the remaining 9 can be anything. Alternatively, if there are 9 digits, the first digit doesn't have to be 0.
- Searching for the email addresses of government employees, where the pattern is: before the "@" symbol, any combination of English letters, dots, or numbers is allowed; after the "@", it can be any organization name, but it must end with .go.th.
- Searching for Thai-language organizational abbreviations, which follow the pattern of 2 to 4 Thai letters followed by a dot (.)

Symbols in regular expressions

- Literal characters
- Metacharacters \w \W \d \s \S \b ^ $.
- Grouping characters [ ] | [^]
- Quantifiers + * ? {}

Literal characters

- Literal characters can be any character on the unicode chart, which encodes the mapping between byte strings and character
- Backslash escape \ is put in front of \ + * ? ^ $ ( ) [ ] { } | .
  - example: www\.chula\.ac\.th

Thai Unicode

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+0E0x		ก	ข	ฃ	ค	ฅ	ฆ	ง	จ	ฉ	ช	ซ	ฌ	ญ	ฎ	ฏ
U+0E1x	ฐ	ฑ	ฒ	ณ	ด	ต	ถ	ท	ธ	น	บ	ป	ผ	ฝ	พ	ฟ
U+0E2x	ภ	ม	ย	ร	ฤ	ล	ฦ	ว	ศ	ษ	ส	ห	ฬ	อ	ฮ	ฯ
U+0E3x	ะ	◌ั	า	◌ำ	◌ิ	◌ี	◌ึ	◌ื	◌ุ	◌ู	◌ฺ					฿
U+0E4x	เ	แ	โ	ใ	ไ	ๅ	ๆ	◌็	◌่	◌้	◌๊	◌๋	◌์	◌ํ	◌๎	◉
U+0E5x	๐	๑	๒	๓	๔	๕	๖	๗	๘	๙	๚	๛				

Metacharacters

Symbol	What it represents
`.`	Any character
`\w`	Word character: a–z, A–Z, 0–9, and _ (Python: includes characters used in words in other languages)
`\W`	Anything that is not `\w`
`\d`	Digit: 0–9
`\D`	Anything that is not `\d`
`\s`	Whitespace (includes space, tab `\t`, newline `\n`, and carriage return `\r`—as used in Windows newlines)
`\S`	Anything that is not `\s`
`\b`	Word boundary (as defined in English)
`^`	Beginning of a string
`$`	End of a string

Exercise 1

Regex: `\d\d\d-\d\d\d-\d\d\d\d`

Entire string match? Partial match? No match?

1. 123-123-1234
2. 123-123-12345
3. a123-123-12345
4. a123-123-1234

Exercise 2

Regex: `^\d\d\d-\d\d\d-\d\d\d\d$`

Entire string match? Partial match? No match?

1. 123-123-1234
2. 123-123-12345
3. a123-123-12345

Exercise 3

Regex: `\w\.\w\.\w\.`

Entire string match? Partial match? No match?

1. U.S.A.
2. ABC
3. พ.ร.บ.

Character grouping

Regex	Matches	Examples
[aei]	a or e or i	b[aei]d matches bad bed bid
[^abc]	one character that is not a b or c	b[^aei]d matches bbd bcd bdd …
aaa|bb|c	aaa or bb or c	bea|ook matches bea ook
[b-d]	b or c or d	[b-d]ad matches bad cad dad
[0-3]	numeral between 0 and 3 (inclusive)	1[0-2] matches 10 11 12
[a-z0-9]	one character in range a-z or in range 0-9	a[a-z0-9] matches aa ab … a0 a1 …
[ก๊-]	one character in range ก๊-	

Thai Unicode

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+0E0x		ก	ข	ฃ	ค	ฅ	ฆ	ง	จ	ฉ	ช	ซ	ฌ	ญ	ฎ	ฏ
U+0E1x	ฐ	ฑ	ฒ	ณ	ด	ต	ถ	ท	ธ	น	บ	ป	ผ	ฝ	พ	ฟ
U+0E2x	ภ	ม	ย	ร	ฤ	ล	ฦ	ว	ศ	ษ	ส	ห	ฬ	อ	ฮ	ฯ
U+0E3x	ะ	◌ั	า	◌ำ	◌ิ	◌ี	◌ึ	◌ื	◌ุ	◌ู	◌ฺ					฿
U+0E4x	เ	แ	โ	ใ	ไ	ๅ	ๆ	◌็	◌่	◌้	◌๊	◌๋	◌์	◌ํ	◌๎	◉
U+0E5x	๐	๑	๒	๓	๔	๕	๖	๗	๘	๙	๚	๛				

Quantifiers

Quantifiers follows a character/metacharacter/character set to indicate quantities.

Symbol	What it represents
`+`	At least one of (>= 1)
`*`	Optional or at least one of (>= 0)
`?`	Optional or one of (0 or 1)
`{n}`	Exactly n of
`{m,n}`	At least m but at most n (>= m and <= n)
`{n,}`	At least n (>=n)
`{,n}`	At most n (<= n)

Exercise 4

Regex: `\w{3,}\.\w@chula\.ac\.th`

Entire string match? Partial match? No match?

1. attapol.t@chula.ac.th
2. te.t@chula.ac.th
3. 6712345253@student.chula.ac.th
4. attapol@chula.ac.th
5. attapol.th@chula.ac.th
6. a.attapol.t@chula.ac.th

Cleaning data

Data: RT @MatichonOnline: “บิ๊กตู่”ลั่นรบ.ทำอะไรยึดกม.ไม่ใช่ติดคุกแล้วหนี เล่นมุกพรรคร่วม “พลังปชป.ภูมิใจไทย” https://t.co/9nmOBJnhrq via @มติชนอ…

Cleaned data: “บิ๊กตู่”ลั่นรบ.ทำอะไรยึดกม.ไม่ใช่ติดคุกแล้วหนี เล่นมุกพรรคร่วม “พลังปชป.ภูมิใจไทย”

- What regexes do we need to write?

Three regexes

```
import re
def clean_tweet(tweet):
    # Remove RT @username
    tweet = re.sub(r'^RT @\w+: ', '', tweet)
    # Remove URL that begins with http
    tweet = re.sub(r'https?://\S+', '', tweet)
    # Remove via @username
    tweet = re.sub(r' via @\w+$', '', tweet)
    return tweet
```

Recap: Regular expression

- Regex is a way to express patterns of characters that we want to match in a string.
- Regex is often used in NLP to clean language data or extract only the information that we need. It's not the sexiest thing to do but it is a very important step.

Issues with text data

- Noise from the data collection channel
- **Variation of languages (sociolinguistic factors)**
- Complexity of language

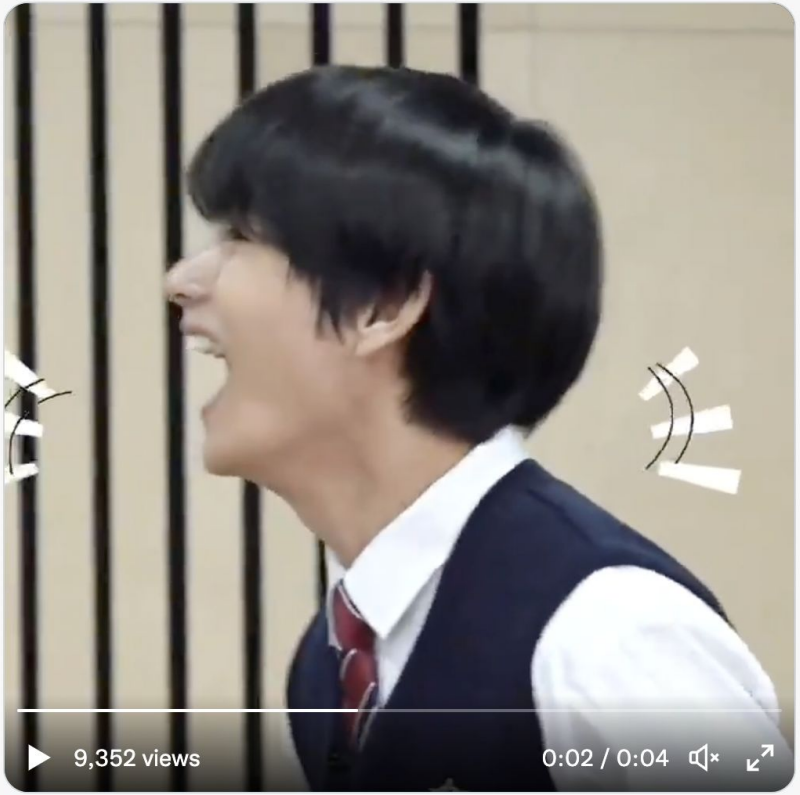
Data on social media platform

- Social media such as Twitter, Facebook, and Instagram, is a data stream which provides cheap and up-to-date opinions of the public.
- We observe its unique context for language variation due to their informal and interactive nature.

ωїηтєr ßєar⁷ 🧸 ✧ vibє @RararabbitJ · Jan 12

หัวเราะต้อนรับเช้าที่ฉดใฉ 😂

I vote for #BTSArmy as #BestFanArmy of @BTS_twt #iHeartAwards 2023.



From **taehourly archive**

Black_KK🖤 @BLACK_KK16 · Jan 7

แบบนี้คือรัยอ่ะ จะบอกว่าไม่จุ๊บ ให้เห็นหรา

#Chaelisa

From **Chaelisa1127**

What it means for NLP?

- How do we know the meaning of these words?
  - ฉดใฉ
  - หรา
  - อาราย
  - น้าาาาาาาา
  - ซีรี่ย์

Issues with text data

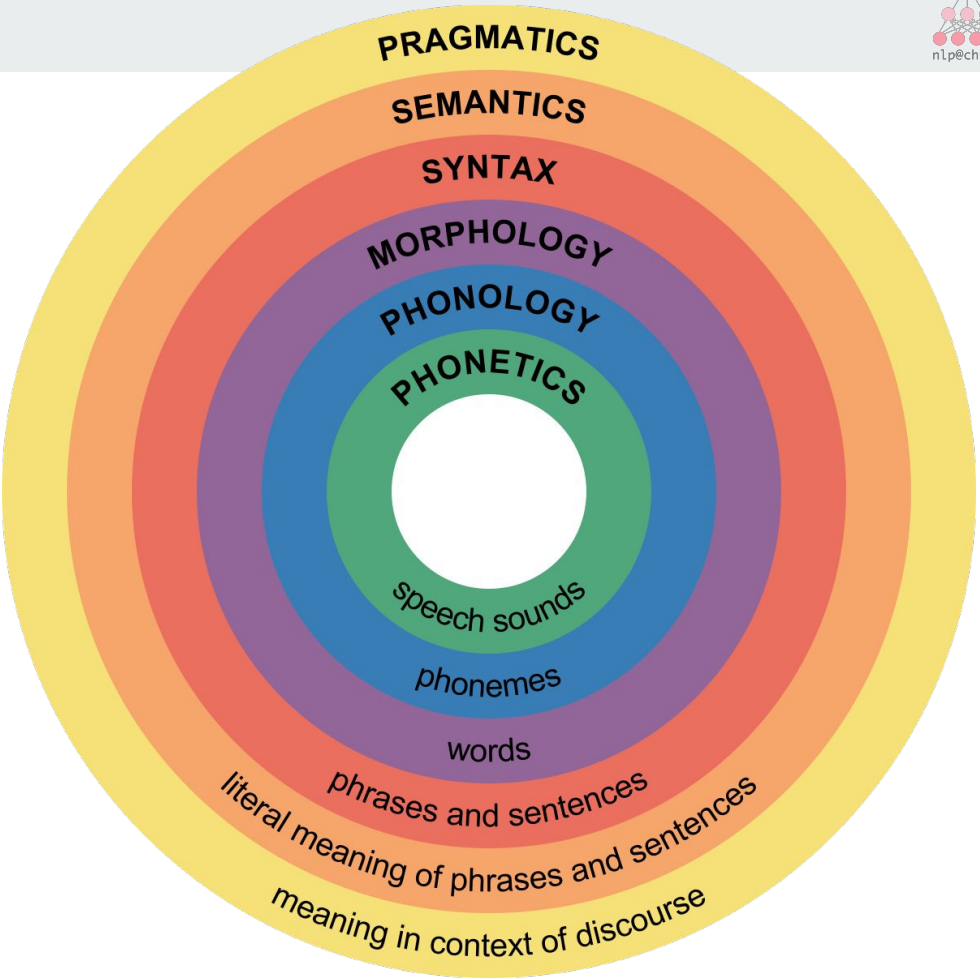
- Noise from the data collection channel
- Variation of languages (sociolinguistic factors)
- **Complexity of language**

Complexity of Language

- Language has structures. To analyze language data, we must analyze its structure
  - Discourse -- a list of sentences
  - Syntax -- sentence and phrase structure
  - Morphology -- structure of a word

Linguistic Structure

Computational linguists study languages by creating models for languages at all levels of analyses



Common linguistic analysis

- In NLP, we call this process linguistic preprocessing.
  - Word segmentation
  - Sentence segmentation
  - Morphological analysis/Normalization

Common linguistic analysis

- In NLP, we call this process linguistic preprocessing.
  - **Word segmentation**
  - Sentence segmentation
  - Morphological analysis/Normalization

Word Segmentation

- Rule-based word segmentation
- Dictionary-based system with maximal matching algorithm e.g. the 'newmm' engine on pythainlp
- Machine learning system e.g. 'deepcut' 'oskut' 'attacut' on pythainlp
- Subword tokenization

How many words are there in each string?

- สวัสดีครับ
- ตัดหญ้า
- ตัดใจ
- คิดถึง
- คิดมาก
- เดินทาง
- สร้างทาง
- ตู้เย็น
- ตะกร้าผ้า
- ขวดแก้ว
- หมอฟัน

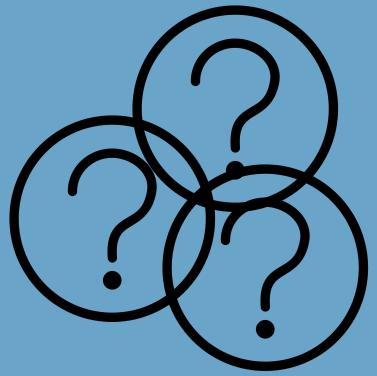
Rule-based word segmentation

- Segment "Got a long list of ex-lovers, they'll tell you I'm insane (Yeah)"
- What are the regular expression rules for the segmentation?

Word Segmentation

- Rule-based word segmentation
- Dictionary-based system with maximal matching algorithm e.g. the 'newmm' engine on pythainlp
- Machine learning system e.g. 'deepcut' 'oskut' 'attacut' on pythainlp
- Subword tokenization

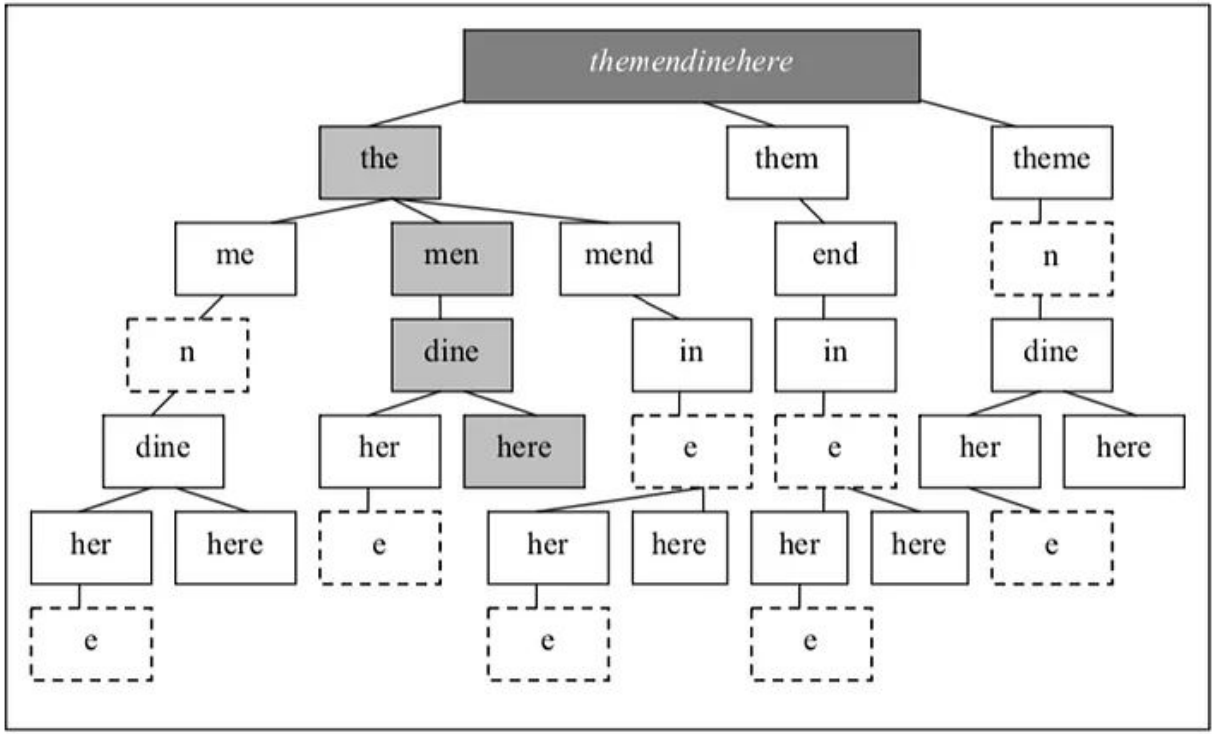
Segment this:
themendinehere



Two ways of dictionary-based segmentation

- Longest-match-first algorithm identifies the longest substring from the start of the input that exists in the dictionary. It processes the text from left to right, consuming and matching substrings step by step without backtracking.
- Longest matching algorithm or maximal matching -- find the segmentation that maximizes the length of substrings that match to the dictionary.

Possible segmentation



Maximal Matching Algorithm

- You need a good dictionary: good coverage but not too broad. What does this mean?
- Generate all possible segmentations and select the combination with the fewest words (each word must be maximally long)

Example

ป้ายกลับรถ

- ป้า|ยก|ลับ|รถ
- **ป้าย|กลับ|รถ**

เดินหามเหสี

- เดิน|หาม|เห|สี
- **เดิน|หา|มเหสี**

อรุ่มเจ๊าะ

หวัดดีครัชชช

Algorithm

- Find all the prefixes that are in the dictionary
  - If found, cut that prefix out and segment the rest
  - If not found, cut out characters until you can find a prefix that is in a dictionary

Your turn

- Assume a standard Thai dictionary. What would the segmentation result for the following sentences:
  a. คำอธิบายบนพรีเซนต์ผิดเยอะจนบอสเขม่น
  b. อาจารย์สั่งโปรเจคที่ต้องคำนวนหาผลลัพธ์
  c. กินยาโคเดอีนแล้วอาจช็อกได้
- Compare your results with

```
import pythainlp
pythainlp.word_tokenize()
```

Word Segmentation

- Rule-based word segmentation
- Dictionary-based system with maximal matching algorithm e.g. the 'newmm' engine on pythainlp
- Machine learning system e.g. 'deepcut' 'oskut' 'attacut' on pythainlp
- Subword tokenization

Machine Learning system

- Machine learning is a family of algorithms that learn certain tasks from data
  - Segment a large amount of text by hand → training samples
  - Use a machine learning model to learn from training samples

Machine learning is very accurate

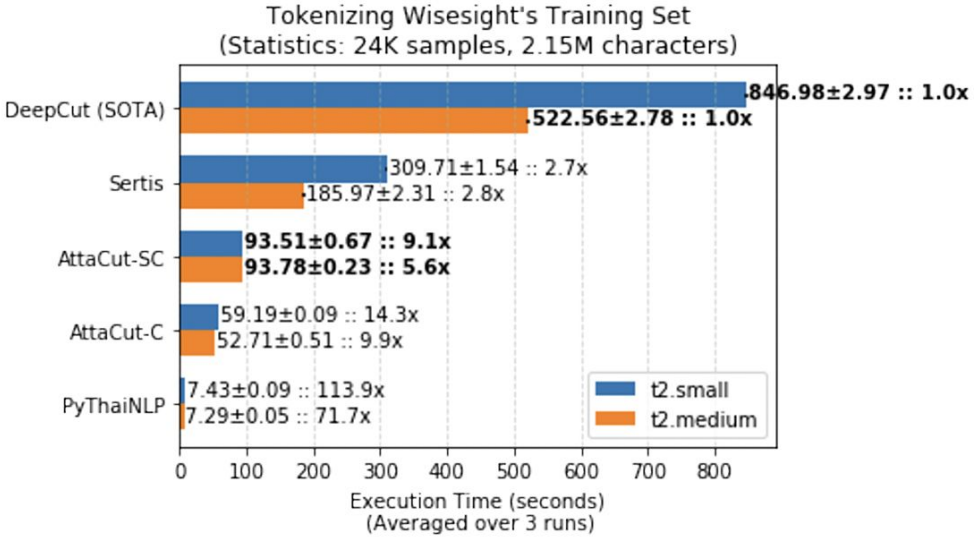
The best accuracy rates are around 79%-95%

Method	Dataset (WL_{F_1})		
	In-Domain	Out-Domain	
	BEST-2010	Wisesight	TNHC
Previous work			
Dictionary-based	71.18%	78.97%	72.70%
DeepCut	94.46%	84.45%	78.17%
Ours			
BiLSTM			
(CH)-BI	95.05%	85.85%	79.31%
(CH+SY)-BI	**95.59%**	**86.15%**	78.70%
-CRF(SY)-BI	95.51%	86.10%	**79.89%**
ID-CNN			
(CH)-BI	94.31%	85.80%	79.22%
(CH+SY)-BI	95.45%	**86.43%**	**79.87%**
-CRF(SY)-SchA*	**95.60%**	86.15%	79.64%

Chormai et al., 2021

Accuracy comes at a cost

Machine learning is at least 10x slower than maximal matching.



Chormai et al., 2021

Word Segmentation is very easy

```
import pythainlp, attacut
test_sentence = "ระฆังดีไม่ตีก็ดัง"

tokens = pythainlp.word_tokenize(test_sentence, engine="attacut")
```

```python
pythainlp.word_tokenize('ระฆังดีไม่ตีก็ดัง แต่คนดีไม่ต้องตีมันก็ดัง', engine='attacut')
```

```
['ระฆัง',
 'ดี',
 'ไม่',
 'ตี',
 'ก็',
 'ดัง',
 ' ',
 'แต่',
 'คน',
 'ดี',
 'ไม่',
 'ต้อง',
 'ตีมัน',
 'ก็',
 'ดัง']
```

```
[5] pythainlp.word_tokenize('พระมหาไพรวัลย์ถึงกับต้องบอกให้เจเจยิ้มแบบอรุ่มเจ๊าะอีกรอบ', engine='attacut')
```

```
['พระมหาไพรวัลย์',
 'ถึง',
 'กับ',
 'ต้อง',
 'บอก',
 'ให้',
 'เจเจยิ้ม',
 'แบบ',
 'อรุ่มเจ๊าะ',
 'อีก',
 'รอบ']
```

```python
import nltk
nltk.download('punkt')
```

```python
doc = nltk.word_tokenize("Apple is looking at buying U.K. startup for $1 billion")
doc
```

```
['Apple',
 'is',
 'looking',
 'at',
 'buying',
 'U.K.',
 'startup',
 'for',
 '$',
 '1',
 'billion']
```

Subword Tokenization

- Motivation:
  - Modern NLP models can tolerate bad word segmentation.
  - In recent years, we try to build an AI/NLP model that works for all tasks and all languages.
  - We have a massive amount of data to process

Subword Tokenization Algorithm

- Three common algorithms:
  - Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
  - Unigram language modeling tokenization (Kudo, 2018)
  - WordPiece (Schuster and Nakajima, 2012)
- All have 2 parts:
  - A token learner that takes a raw training corpus and induces a vocabulary (a set of tokens).
  - A token segmenter that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,…, a, b, c, d….}

Repeat:

- Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
- Add a new merged symbol 'AB' to the vocabulary
- Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until k merges have been done.

BPE token learning algorithm

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens til k times
$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
$V \leftarrow V + t_{NEW}$ # update the vocabulary
Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Subword tokenization

- Most subword algorithms are run inside space-separated tokens
- So we commonly first add a special end-of-word symbol _ before space in training corpus
- Next, separate into letters.

BPE token learner example

Classic corpus:

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

{_, d, e, i, l, n, o, r, s, t, w}

corpus		vocabulary
5	l o w _	_, d, e, i, l, n, o, r, s, t, w
2	l o w e s t _	
6	n e w e r _	
3	w i d e r _	
2	n e w _	

Merge e r to er

corpus		vocabulary
5	l o w _	_, d, e, i, l, n, o, r, s, t, w, er
2	l o w e s t _	
6	n e w er _	
3	w i d er _	
2	n e w _	

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge er _ to er_

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

Merge n e to ne

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_, ne

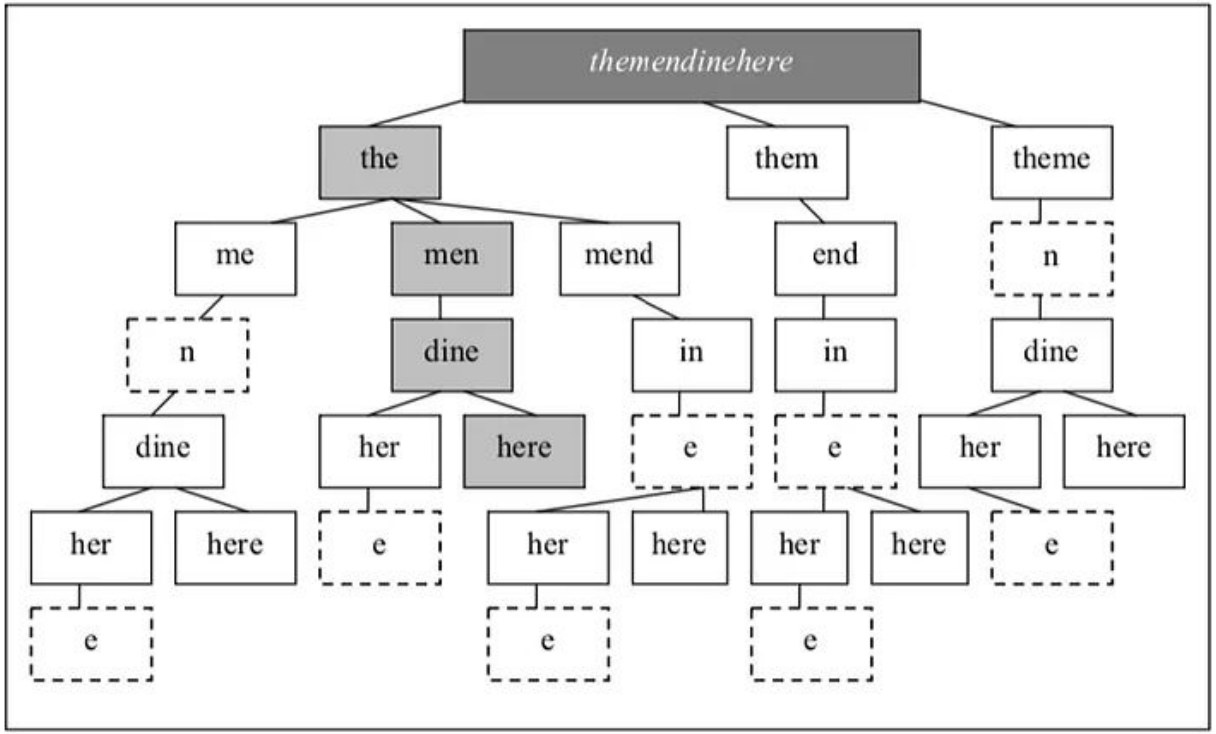
The next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

Token learner + Token segmenter

- A token learner returns a vocabulary and is discarded.
- A token segmenter is a dictionary-based segmenter, but we use longest-matching algorithm, which is a greedy algorithm.
    - Why?

Longest Matching Algorithm



How does it work in practice?

- The final vocabulary usually includes frequent words
- Frequent subwords are often morphemes like -est or –er
- A morpheme is the smallest meaning-bearing unit of a language
    - unlikeliest has 3 morphemes un-, likely, and -est

Summary - word segmentation/tokenization

Method	Key component	Pros	Cons
Rule-based	Regexes	Simple and accurate	Not applicable at all to languages without word boundaries
Dictionary-based	Lexicon	Accurate, fast, and customizable	Not generalizable
ML-based	Trained ML model	Best option for languages without word boundaries	Slow and expensive to customize
Subword tokenization	Lexicon trained on a large corpus	Fast	The results are not words

Compared to other challenges, word segmentation is not your biggest worry.

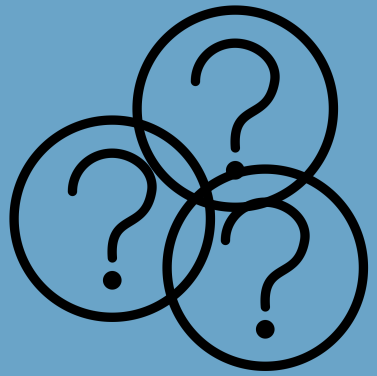
Common linguistic analysis

- In NLP, we call this process linguistic preprocessing.
  - Word segmentation
  - **Sentence segmentation**
  - Morphological analysis/Normalization

Sentence segmentation

- Sentence segmentation is also known as sentence boundary detection. It is a very important task for NLP because it allows us to process a smaller and meaningful chunk of text (instead of the entire long chunk)

How do we do sentence segmentation for English?



Regular expression

- ! ? are very good boundaries, but . is very ambiguous
  - Dr. Attapol doesn't work at Google Inc.
  - I think … the accuracy is 10.5%.
- Tokenize and use some complicated set of rules and abbreviation dictionary.

Machine Learning (Kiss and Strunk, 2006)

- Given a large amount of data, find the collocations or words that appear together a lot. Those should not be separated into two sentences.
- Find words that are at the beginning of the text (hence beginning of the sentence) a lot.

Accuracy rates are very high

It turns out to be effective for many languages that ambiguously mark their sentence boundaries.

Corpus	Error (<S>) (%)	Prec. (<S>) (%)	Recall (<S>) (%)	F (<S>) (%)
B. Port.	1.11	99.14	99.72	99.43
Dutch	0.97	99.25	99.72	99.48
English	1.65	99.13	98.64	98.89
Estonian	2.12	98.58	99.07	98.83
French	1.54	99.31	99.08	99.19
German	0.35	99.69	99.93	99.81
Italian	1.13	99.32	99.49	99.41
Norw.	0.81	99.45	99.68	99.56
Spanish	1.06	99.66	99.23	99.45
Swedish	1.76	98.82	99.36	99.09
Turkish	1.31	99.40	99.24	99.32
Mean	1.26	99.25	99.38	99.31
SD	0.49	0.33	0.38	0.29

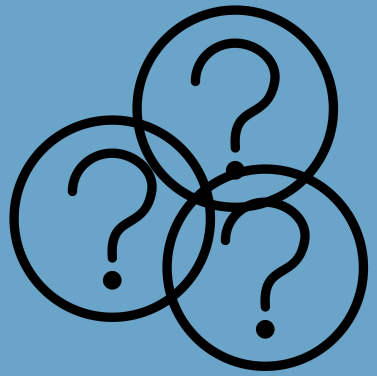
Sentence segmenting

```
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import
PunktSentenceTokenizer

text = "this is a sentence. And this is
another one! are you enjoying it?"
tokenizer = PunktSentenceTokenizer()
sentences = tokenizer.tokenize(text)
print(sentences)
```

```
['this is a sentence.', 'And this is
another one!', 'are you enjoying
it?']
```

How do we do sentence segmentation for Thai?



Thai Sentence Segmentation

- Sentence boundary detection in Thai is quite difficult because we only have spaces which are highly ambiguous. The most viable approach is to use machine learning.
- No existing good library. Pythainlp is quite bad at sentence segmentation.

Machine Learning for Thai SS

The results are quite impressive. But the tools are not available…

Model	Orchid		
	precision (%)	recall (%)	F1 (%)
(a) POS-trigram [12]	74.4	79.8	77.0
(b) Winnow [13]	92.7	77.3	84.3
(c) ME [11]	86.2	83.5	84.8
(d) CRF (Thai baseline) [14]	94.7	89.3	91.9
ine (e) Bi-LSTM-CRF [20]	92.1	89.7	90.9
(f) + local	93.1	91.7	92.4
(g) + local + distant	**93.5**	**91.5**	**92.5**
(h) + local + distant + CVT	-	-	-
(i) + local + distant + CVT + ELMo	-	-	-

Saetia et al., 2021

Common linguistic analysis

- In NLP, we call this process linguistic preprocessing.
  - Word segmentation
  - Sentence segmentation
  - **Morphological analysis/Normalization**

Verb form and nouns

- seek (infinitive)
  - seeks (present)
  - sought (past form)
  - seeking (participle form)
  - seeker (noun)
- match (noun)
  - matches (plural)

Stemming and lemmatization

- Lemma is the base or dictionary form of a word, determined by its meaning and grammatical context.
- Stem is the root form of a word obtained by removing prefixes and suffixes, without necessarily forming a valid word.
- Which one is harder to do?

Word	Lemmatized	Stemmed
seeks	seek	seek
sought	seek	sought
seeking	seek	seek
seeker	seeker	seek

Conclusion

- Texts are a bunch of strings that encode language data.
- A corpus is a collection of texts.
- To create a corpus or do useful things with texts, we need to
  - clean the data and select the parts relevant to our use
  - perform automatic linguistic preprocessing: word segmentation (and possibly sentence segmentation and word normalization)